

Guía de Ejercicios 1

Introducción a la programación en Assembly

Introducción

Se aconseja realizar los ejercicios primero en C y luego en Assembly, verificando posteriormente que el algoritmo funcione correctamente. Se recomienda que los ejercicios se hagan de manera consecutiva y que se realicen comentarios de manera prolija y ordenada. Implementar todas las funciones de Assembly en un mismo archivo .S y las funciones en C en un mismo archivo .c.

Ejercicio 1

Implementar una función que realice el producto de un vector con un escalar. Utilizar el siguiente prototipo para la función:

```
void productoEscalar32(uint32_t *vectorIn, uint32_t *vectorOut, uint16_t longitud, uint16_t  
escalar);
```

Ejercicio 2

Modificar la función del ejercicio anterior para trabajar con datos de 16 bits y que el resultado sature en 12 bits. Utilizar el siguiente prototipo para la función:

```
void productoEscalar12(uint16_t *vectorIn, uint16_t *vectorOut, uint16_t longitud, uint16_t  
escalar);
```

Ejercicio 3

Realizar una función que borre un campo de bits de un dato de 32 bits (ver Figura 1). Utilizar el siguiente prototipo para la función:

```
uint32_t bitfield_clear(uint32_t dato, uint32_t ancho, uint32_t inicio);
```

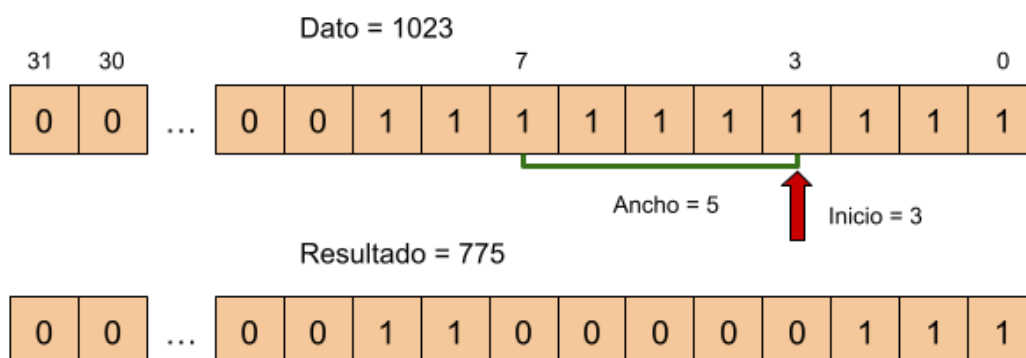


Figura 1: Ejemplo de función que borra un campo de bits.

Ejercicio 4

Modificar el código desarrollado en el ejercicio anterior para que se puedan invertir los bits del campo seleccionado (función toggle, ver Figura 2). Utilizar el siguiente prototipo para la función:

```
uint32_t bitfield_toggle (uint32_t dato, uint32_t ancho, uint32_t inicio);
```

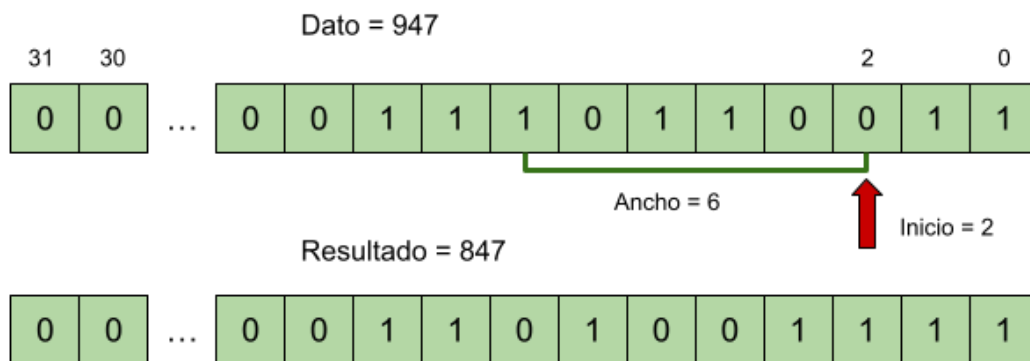


Figura 2: Ejemplo de función que invierte (toggle) los bits de un campo de bits.